PHY4605 Computational Methods in Physics
Learning Note | Week 4

Parameter Sweeps: Compare one controlled change and interpret the graph

Use one familiar spring model to turn parameter variation into a controlled computational experiment.

Physical question. If the same forces are applied to springs with different stiffness values, how can we compare their extensions fairly and explain the trend using both the graph and Hooke's law?

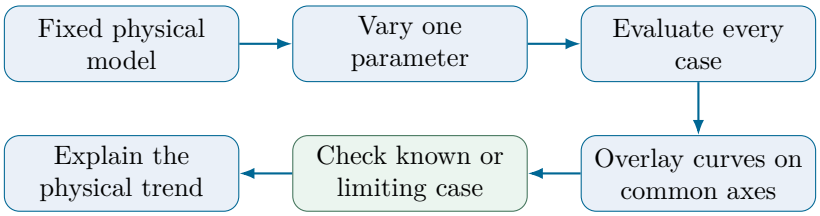
Learning outcomes

1. design a controlled sweep that varies one physical parameter while keeping the model and other inputs fixed;
2. evaluate and overlay several parameter cases on common labelled axes; and
3. validate a known or limiting case and explain the trend in physical language.

How to use this note. *Follow the numbered route from a controlled model change to a fair graph, validation, and physical interpretation. Pause at each checkpoint and predict before reading the result.*

1. A parameter sweep is a controlled experiment

A parameter sweep repeats the *same model* for several values of one chosen parameter. Here, applied force is the independent variable and the same force array is used for every curve; the equation, assumptions, and other inputs stay fixed.



Common mistake. Changing two parameters together confounds their effects: even a smooth graph cannot show which change caused the difference.

2. Familiar model: an ideal spring

Within its linear elastic range, an ideal spring follows Hooke's law,

$$F = kx, \qquad \text{so} \qquad x = \frac{F}{k}.$$

In words: *extension equals applied force divided by spring stiffness.* For a fixed force, increasing stiffness must reduce extension.

Symbol	MATLAB name	Meaning and unit
F	`F_N`	applied force, N; common input array
k	`k_Npm`	spring stiffness, N m^{-1}; swept parameter
x	`extension_m`	spring extension, m; calculated output

Use the common force array $F = 0$ to 10 N and three stiffnesses: 50, 100, and 200 $\mathrm{N\,m^{-1}}$. Use 100 $\mathrm{N\,m^{-1}}$ as the reference (baseline) case.

> **Prediction checkpoint.** At the same positive force, the 50 $\mathrm{N\,m^{-1}}$ spring should extend the most and the 200 $\mathrm{N\,m^{-1}}$ spring the least. At $F = 0$, every ideal spring must have $x = 0$.

3. Decide what changes and what stays fixed

Before writing code, make the experimental control explicit.

Deliberately changed	Held fixed
Spring stiffness k: 50, 100, 200 N/m	Hooke model $x = F/k$, force array $0 : 0.5 : 10$ N, ideal-linear assumption, plotting axes, and output definition

This control statement is part of the scientific result. It tells the reader what can legitimately explain the difference between curves.

4. Pseudocode before MATLAB

1. define one common applied-force array;
2. define a small array of stiffness values and identify the reference case;
3. for each stiffness, calculate $x = F/k$ using the same force values;
4. store one extension curve for each stiffness;
5. overlay all curves using the same axes and a legend;
6. compare the same force across cases;
7. check the zero-force limiting case; and
8. explain why the curves differ using the physical equation.

5. Build a small parameter array

The force array contains the repeated physical inputs. The stiffness array contains the cases to compare.

Listing 1: Define the common force array, swept stiffness values, and reference case.

```
F_N = 0:0.5:10;       % common applied-force array (N)
k_Npm = [50 100 200]; % swept spring stiffness values (N/m)
baseline_k_Npm = 100; % reference case (N/m)
```

The comparison uses only three stiffness values because the goal is to make the trend visible and traceable. A dense sweep is not automatically a better experiment.

6. Evaluate the same model for every case

Preallocate one matrix row for each stiffness. The loop changes only the selected stiffness value; the force array and equation remain unchanged.

Listing 2: Evaluate extension for each stiffness using the common force array.

```
extension_m = zeros(length(k_Npm),length(F_N));
for case_id = 1:length(k_Npm)
    extension_m(case_id,:) = F_N./k_Npm(case_id);
end
```

Row 1 stores the 50 $\mathrm{N\,m^{-1}}$ case, row 2 the reference case, and row 3 the 200 $\mathrm{N\,m^{-1}}$ case. At $F = 10$ N,

$$x = \frac{10}{50} = 0.20\,\mathrm{m}, \quad x = \frac{10}{100} = 0.10\,\mathrm{m}, \quad x = \frac{10}{200} = 0.05\,\mathrm{m}.$$

Code-tracing checkpoint. In `extension_m(3,end)`, index 3 selects the $200\,\mathrm{N\,m^{-1}}$ spring and `end` selects the largest stored force, 10 N. The expected value is 0.05 m.

7. Overlay curves so the comparison is fair

All parameter cases should share the same axes. A legend identifies which curve belongs to which stiffness.

Listing 3: Plot all stiffness cases on common applied-force and extension axes.

```
plot(F_N,extension_m.','LineWidth',2)
grid on
xlabel('Applied force, F (N)')
ylabel('Spring extension, x (m)')
title('Hooke''s Law Parameter Sweep: Effect of Spring Stiffness')
legend('k = 50 N/m','k = 100 N/m','k = 200 N/m')
```

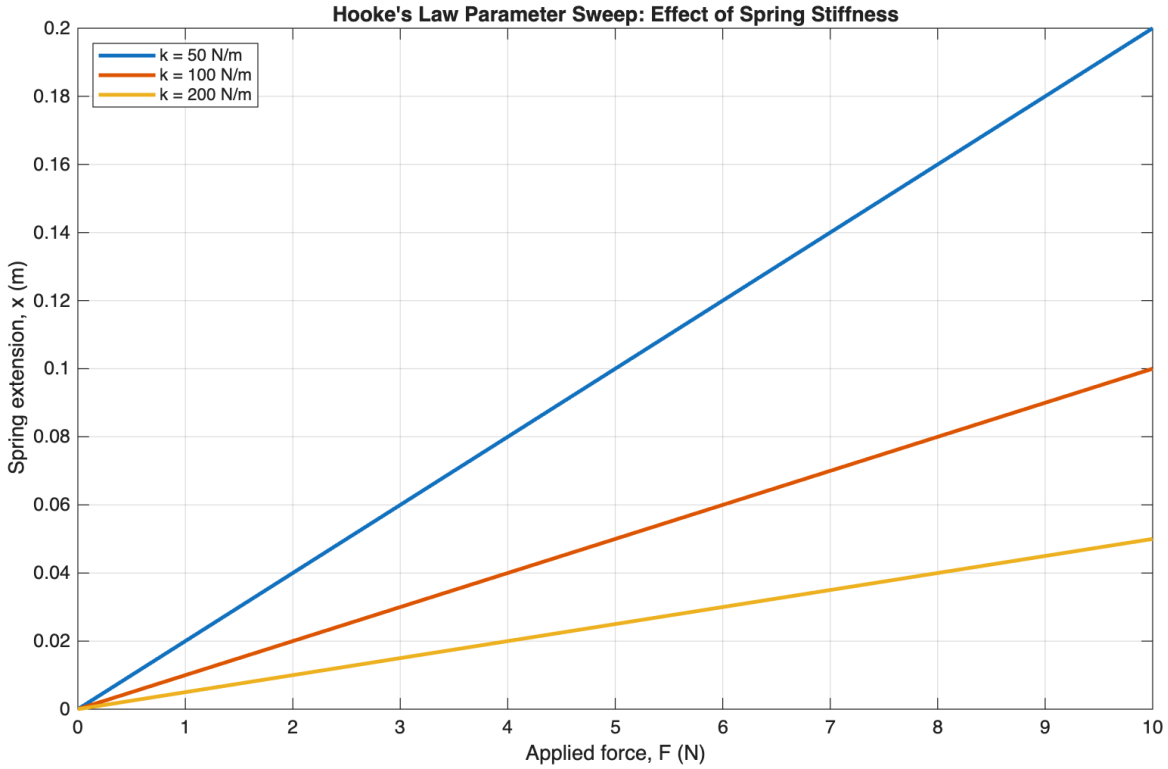


Figure 1: MATLAB-generated parameter sweep for the Hooke model used in this example. Every curve uses the same applied-force array; only spring stiffness changes.

8. Read the graph as physics, not only as geometry

The three curves are straight because x is proportional to F when k is fixed. Their slopes differ because the slope is $1/k$. A smaller stiffness gives a larger slope, so the spring extends more for each additional newton of force. A larger stiffness gives a smaller slope, so the same load produces less deformation.

Graph interpretation rule. Do not stop at "the blue line is steeper." State the physical cause: the smaller stiffness produces more extension at the same force because $x = F/k$.

9. Required validation: zero-force limiting case

At zero applied force, the known limiting value is

$$x(0) = \frac{0}{k} = 0$$

for every positive stiffness. It is known before running the program and tests all swept cases together.

Listing 4: Check the zero-force limiting value for every stiffness case.

```
zero_force_extension_m = extension_m(:,1);
assert(max(abs(zero_force_extension_m)) < 1e-12,...
    'Zero-force limiting-case validation failed.')
```

Validation result. The first stored extension is 0 m for all three stiffness values, so the limiting case passes from a fresh MATLAB session.

10. Diagnose a runnable but physically wrong model

Suppose the model is entered as `x = F.*k`. MATLAB can calculate it, but it no longer represents Hooke's law: the units are wrong and larger stiffness produces a larger output.

Check	**Correct $x = F/k$**	**Incorrect product $F\,k$**
Units	N /(N/m) = m	N(N/m), not m
Stiffness trend	larger k gives smaller x	larger k gives larger value
At $F = 0$	zero	also zero; this check alone cannot expose the formula error

Responsible-AI caution. Generated code can pass a limiting check yet use the wrong equation. Combine the physical model, units, expected trend, and validation evidence.

Guided extension: selected values and two parameters

Read the supplied table to compare outputs at common forces. A supplied two-parameter table broadens the comparison; designing heatmaps is not required in Week 4.

Optional extension: local sensitivity

If useful, read the supplied fixed-force result $\partial x/\partial k = -F/k^2$. Its negative sign formalises the main trend; quantitative sensitivity is optional, not required in Week 4.

Three takeaways

1. Change one named parameter while keeping the model and other inputs fixed.
2. Overlay curves on common labelled axes and compare the same input across cases.
3. Explain the trend with the physical equation, then complete the required limiting-case validation.

Preparation for the practical. Transfer the sweep logic to Ohm's law, vertical motion, or a small-angle pendulum. For each, state changed and fixed inputs, predict the graph, and name an independent check.